

Actividad I. Matriz Descriptiva y Análisis Paradigmático

Matriz Descriptiva de Paradigmas de Programación

Paradigma	Principios Fundamentales	Ventajas	Desventajas	Lenguajes Representativos
Imperativo/ Estructural	Se basa en la ejecución secuencial de instrucciones que modifican el estado del sistema mediante variables y estructuras de control. El programador especifica paso a paso cómo resolver un problema.	Fácil comprensión, control directo sobre la memoria y alta eficiencia computacional.	Mayor dificultad para mantener programas grandes debido a la dependencia del estado y los efectos secundarios.	C, Pascal, Fortran
Orientado a Objetos (POO)	Organiza el software mediante objetos que combinan datos y comportamiento. Se fundamenta en encapsulamiento, herencia, polimorfismo y abstracción.	Favorece la reutilización de código, modularidad y mantenimiento de sistemas complejos.	Puede generar jerarquías excesivamente complejas y mayor consumo de recursos.	Java, C++, C#, Python
Funcional	Considera las funciones como elementos de primera clase. Promueve la inmutabilidad, la transparencia referencial y la reducción de efectos secundarios.	Facilita la concurrencia, mejora la mantenibilidad y favorece el razonamiento matemático sobre el código.	Curva de aprendizaje elevada para programadores acostumbrados al paradigma imperativo.	Haskell, Lisp, F#, Scala
Lógico/Declarativo	El programador define relaciones y reglas, mientras el sistema determina el procedimiento para alcanzar una solución mediante mecanismos de inferencia.	Simplifica problemas de inteligencia artificial, sistemas expertos y procesamiento simbólico.	Menor rendimiento en ciertas aplicaciones de propósito general.	Prolog, Datalog
Concurrente /Basado en Actores	Utiliza entidades independientes denominadas actores que se comunican mediante paso de mensajes sin compartir estado interno.	Reduce condiciones de carrera y facilita sistemas distribuidos y escalables.	Mayor complejidad conceptual para diseñar y depurar aplicaciones concurrentes.	Erlang, Elixir, Akka

Paradigma Imperativo/Estructural

El paradigma imperativo representa uno de los enfoques más antiguos y difundidos en la programación.

- **Gestión del Estado:** Su característica principal es la gestión explícita del estado del sistema mediante variables que pueden modificarse durante la ejecución.
- **Flujo de Control:** El programador controla directamente la secuencia de instrucciones, la asignación de memoria y el flujo de ejecución.
- **Eficiencia y Riesgos:** Esta capacidad ofrece un alto nivel de eficiencia, aunque también incrementa la posibilidad de errores derivados de efectos secundarios y modificaciones inesperadas del estado.

Paradigma Orientado a Objetos (POO)

La programación orientada a objetos surge como respuesta a la complejidad creciente del software.

- **Modelo de Abstracción:** Su modelo organiza el sistema en objetos que encapsulan atributos y métodos relacionados.
- **Pilares Fundamentales:** El encapsulamiento protege los datos internos, mientras que el polimorfismo permite diferentes implementaciones de una misma interfaz.
- **Reutilización de Código:** La herencia y la composición facilitan la extensibilidad del sistema; actualmente, la composición es considerada una alternativa más flexible que la herencia para construir arquitecturas escalables.

Paradigma Funcional

El paradigma funcional se fundamenta en principios matemáticos donde las funciones constituyen el elemento central del programa.

- **Inmutabilidad:** Evita modificaciones directas sobre los datos, reduciendo drásticamente los errores asociados al estado compartido.
- **Transparencia Referencial:** Garantiza que una función produzca siempre el mismo resultado para una misma entrada, eliminando efectos colaterales.
- **Optimización:** Mecanismos avanzados como la evaluación perezosa (*lazy evaluation*) permiten optimizar recursos ejecutando únicamente las operaciones cuando su resultado es estrictamente necesario.

Paradigma Lógico/Declarativo

A diferencia de los enfoques anteriores, la programación lógica se centra en describir *qué* problema debe resolverse en lugar de especificar el *cómo*.

- **Mecanismo Operativo:** Mediante la definición explícita de relaciones, hechos y reglas, el sistema aplica procesos internos de unificación y resolución para deducir soluciones de forma autónoma.
- **Campos de Aplicación:** Este enfoque resulta especialmente útil en inteligencia artificial, sistemas expertos y motores de inferencia donde las relaciones lógicas predominan sobre los procedimientos secuenciales.

Paradigma Concurrente / Basado en Actores

El crecimiento de los sistemas distribuidos y los procesadores multinúcleo ha impulsado la adopción de modelos concurrentes basados en actores.

- **Aislamiento de Estado:** Cada actor posee su propio estado privado y se comunica exclusivamente mediante el paso de mensajes asíncronos.
- **Mitigación de Errores:** Este aislamiento elimina de raíz los problemas tradicionales de la concurrencia, como los bloqueos mutuos (*deadlocks*) y las condiciones de carrera. Como resultado, es una alternativa sumamente eficiente para servicios en la nube y sistemas de alta disponibilidad.

Convergencia Multiparadigma

La evolución de los lenguajes de programación ha demostrado que ningún paradigma resulta suficiente para resolver todos los problemas computacionales de forma óptima. Por esta razón, los lenguajes modernos incorporan características de múltiples frentes simultáneamente:

- **Python y JavaScript:** Permiten entrelazar enfoques imperativos, funcionales y orientados a objetos en un mismo archivo de código.
- **Java y Scala:** Han integrado expresiones lambda y herramientas de programación funcional sobre arquitecturas nativamente orientadas a objetos.

Impacto en el Mercado: Esta convergencia responde a las demandas de la industria actual, donde los sistemas requieren flexibilidad de diseño (POO), seguridad en procesamiento paralelo (Funcional) y eficiencia de bajo nivel (Imperativo). En consecuencia, el dominio de la diversidad paradigmática constituye una competencia esencial para el ingeniero informático moderno.